面向 CODESYS 平台的 ST 代码自动生成系统

——GenAI 课程大作业

姓名

浙江大学

2026 年 11 月 10 日

目标、交付与评价取向

目标

输入自然语言需求 + 接口表 + LTL 规约；输出可直接在 CODESYS 编译并运行的 **ST**

难点不在“像不像”

- 语法与工程结构：DUT、Standard 库、调用图
- 行为语义：状态机、定时器、边沿、优先级
- 评测可信：必须**真实执行**

交付

数据与规约（30 题 / 75 条 LTL / 54 组用例）· 自研 Agent harness · CODESYS 编译 + 仿真服务 · 统一评测管线

四种设置

{Codex CLI, 自研} × {DeepSeek, Qwen}，各 30 题，统一 5 轮修复预算，`tool_error` 已清零

要点 判定必须来自**真实 CODESYS 编译 + 仿真运行**；自研解释器会引入“不支持某语法”的假失败，文本相似度则分不清“差一拍”.

数据集与质量基线

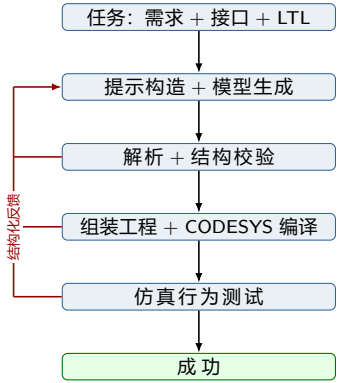
维度	数量
总题数	30（公共 20 + 自建 10）
难度	easy 9 / medium 12 / hard 9
形式化规约	75 条（2.5 条/题）
测试用例	54 组、199 个扫描步

每题包含：`FUNCTION_BLOCK` 接口（含 DUT）+ 中英需求 + LTL + 行为用例 + 可编译参考实现

- **自建题**：交通灯、水箱、自动门、分拣、主备泵、报警闭锁、星三角、停车场、恒压供水、电梯——均为真实 PLC 场景
- **多样性**：离散制造/流程工业/电机/安全/队列/字符串；含 DUT 3 题、`VAR_IN_OUT` 4 题
- **规约**：互斥、响应、保持、范围不变量、错误码
- **用例**：正常流程、边界、优先级冲突、错误处理；**每步写明测试目的**并关联到 LTL 规约

要点 先钉死“题可做、用例对”：参考实现 **30/30** 编译且 **30/30** 通过自己的用例（首轮自检曾暴露 16 处用例缺陷），才谈模型通过率.

架构：单 Agent 反馈闭环



- 一次生成完整工程，每轮整体重写，不做局部补丁
- **结构校验前置**：格式/接口错误不消耗 CODESYS 冷启动
- **上下文**：仅保留最近 2 轮；错误按 (code, text) 去重
- **终止**：通过 / 5 轮 / 代码无变化 / 失败签名重复 / 工具故障
- 测试用例**对模型不可见**，仿真基于 CODESYS 官方运行时

要点 两道闸门：零成本预校验 → 编译（约 13 s）→ 仿真（约 24 s）；任何一步失败都转成**结构化反馈**（错误码 + 行列，或失败的用例/步 + 期望值 + 实测值）回到模型.

实验设置与公平性

	deepseek-v4.1-flash-1017	qwen3.7-flash
现有框架（Codex CLI）	实验 1	实验 2
自研 harness	实验 3	实验 4

- Codex CLI 包装为与自研相同的 `chat(messages)` 客户端：**提示、反馈文本、轮次上限、编译测试管线完全一致**
- 差异只在生成组织方式（Codex 自带 agent 循环与工具上下文）
- 同一 DMXAPI 网关、同一模型快照、同一 CODESYS 模板与 worker
- **指标**：首次编译、最终编译、测试通过率、token、墙钟时间、轮次
- Codex 的工具调用开销计入 token 与耗时——这是“现有框架”的真实成本

要点 除“生成组织方式”外全部对齐，因此结果差异只能归因于框架本身，而非提示、预算或工具链.

总体结果：DeepSeek 打平，Qwen 拉开差距

设置	首次编译	最终编译	测试通过	平均轮次	平均耗时
自研 + DeepSeek	96.7%	100.0%	**83.3%**	1.77	163 s
自研 + Qwen	76.7%	93.3%	**66.7%**	2.53	295 s
Codex + DeepSeek	100.0%	100.0%	**83.3%**	1.70	342 s
Codex + Qwen	16.7%	53.3%	**33.3%**	4.00	431 s

83.3% / 83.3%
DeepSeek 下两框架测试通过率**完全持平**

66.7% vs 33.3%
Qwen 下自研**高出 33 个百分点**

96.7% vs 100%
首轮编译：自研靠反馈追平最终结果

要点 DeepSeek 下框架不改变质量上限，只改变成本；Qwen 下框架改变质量本身——说明存在**框架 × 模型交互**.

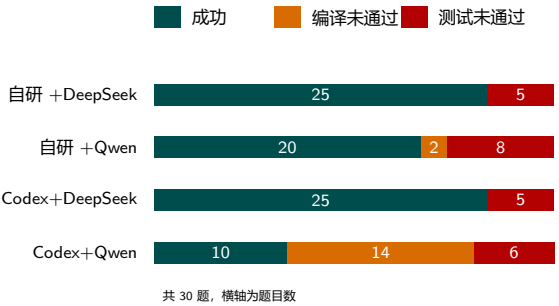
代价：同样的通过率，输入成本差一个量级

设置	输入 tokens	输出 tokens	平均耗时	相对 Codex
自研 + DeepSeek	75,058	383,312	163 s	输入 **13%**、耗时 **48%**
自研 + Qwen	131,255	503,606	295 s	输入 **8%**、耗时 **68%**
Codex + DeepSeek	565,110	521,147	342 s	—
Codex + Qwen	1,632,869	358,524	431 s	—

- 通过率相同（83.3%）的前提下，自研输入 token 为 Codex 的 **1/7**、耗时约 **1/2**
- 差距几乎全在**输入侧**：Codex 的 agent 循环反复累加系统提示、工具定义与历史
- 输出 token 两框架同量级（383k vs 521k），说明“想的量”相当，差的是“看的量”

要点 自研的价值不是“用更聪明的模型”，而是用**更短的上下文**拿到同样的结果：弱模型少看无关内容，也就少犯错.

解释一：差距落在编译层，不是行为层



- DeepSeek 两种框架**编译失败均为 0**：编译闸门已不是瓶颈，扣分全在行为层
- 自研 + Qwen 也只剩 2 题编不过，靠反馈把 28 题送进测试
- Codex + Qwen 有 **14 题最终仍编译失败**：首轮编译仅 16.7%，5 轮预算基本耗在语法上

要点 框架的差距首先是“有没有把代码编译通”：只看编译通过的题，行为通过率为 83.3% / 83.3% / 71.4% / 62.5%——Qwen 下两框架的差距从 **33 个百分点收窄到 9 个百分点**，其余差距来自没编译过的题.

解释二：框架 × 模型交互，Qwen 对框架敏感

模型	自研败	Codex 败	重合	自研独有
DeepSeek	5	5	**4**	1
Qwen	10	20	**10**	**0**

失败题集高度重合：DeepSeek 下两框架只在 1 题上各自不同；Qwen 下自研的 10 题失败**全部**包含在 Codex 的 20 题失败里，Codex 多出的 10 题中 6 题是编译失败.

- **错误码**（Codex+Qwen 累计）：C6 140 次（把 := 写成 =、用 !）、C189 95、C9 80、C32 43、C135 41
- **轮次**：18/30 题耗尽 5 轮；首轮编译失败 25/30
- **难度**：Codex+Qwen 在 medium 仅 2/12 通过——中型题语法规模更大，错误更早累积
- 自研的“块格式 + 接口预校验 + 单轮整体重写”更贴合 Qwen 的输出习惯；Codex 的通用 agent 提示反而诱发非 IEC 写法

要点 不能假设“更强的 agent 框架对任意模型都更好”：同一模型换框架，通过率相差 33 个百分点；而 DeepSeek 下换框架质量不变、成本翻 7 倍. **框架与模型需要匹配调优**.

剩余失败在哪里：行为语义，编译反馈管不到

- 状态机题：五工位生产、柔性加工
- 定时/边沿题：焊缝去抖、自动门、交通灯
- 典型症状：“差一拍”、未覆盖优先级
- 3 题在**全部 4 个设置下同时失败**（`public_hard_003/004`、`self_easy_003`），另 3 题在 3 个设置下失败——指向用例与规约的判定强度，而非某个模型的能力

判定强度的缺口（如实标注）

- 驱动只断言 `expect` 中的输出：12 个扫描步无断言
- 34 个布尔输入从未置 TRUE，分布在 9 道题
- 规约未接入 PLCVerif；每题仅采样 1 次

因此上表通过率是**“现有用例下”**的通过率，严格化后可能下降.

要点 编译反馈已解决“能编译”，剩下的价值在**更强的行为规约/验证**（PLCVerif）与更严的用例覆盖，而不是继续堆编译轮次.

结论与后续

1. 建成 30 题数据集，逐题 LTL 规约、测试用例（含逐步测试目的）与参考实现；参考实现 30/30 通过编译与仿真，形成可复现基线
2. 自研 harness 在 DeepSeek 下达到与 Codex 相同的测试通过率（83.3%），输入 token 约为其 1/7、耗时约 1/2；在 Qwen 下显著更优（66.7% vs 33.3%）
3. 差距主要落在编译层与上下文成本，而非“模型想得不够”；剩余失败集中在行为语义

如实标注的局限

- 规约未接入 PLCVerif（选做项，未做）；每题仅采样 1 次，存在方差
- 12 个扫描步无断言、34 个布尔输入从未置 TRUE，通过率是“现有用例下”的通过率
- 下一步：接入 PLCVerif、补强用例覆盖、多平台验证

要点 把“能不能编译、跑得对不对”做成**结构化反馈**，弱模型也能被推上来；但框架与模型必须匹配——没有对所有模型都更好的框架.